Compliance Anomaly Detection in Higher Education Academic Data Using Rule-Based and Machine Learning Methods

1st Benny Leonard Enrico Panggabean
Informatics Engineering, Faculty of Engineering
Universitas Hasanuddin
Makassar, Indonesia
bendo01@gmail.com
ORCID: 0000-0001-6805-6463

2nd Amil Ahmad Ilham
Informatics Engineering, Faculty of Engineering
Universitas Hasanuddin
Makassar, Indonesia
amil@unhas.ac.id
ORCID: 0000-0002-4755-6415

3rd Novy N. R. A. Mokobombang
Informatics Engineering, Faculty of Engineering
Universitas Hasanuddin
Makassar, Indonesia
novy@unhas.ac.id
ORCID: 0000-0001-5706-939X

DRAFT

***Abstract*—Higher education institutions in Indonesia report their academic data to the national database PDDIKTI, and records that violate reporting rules weaken quality assurance and accreditation. This paper studies the detection of compliance anomalies in PDDIKTI data alone, without any institutional source system. Explicit validation rules taken from the Feeder PDDIKTI user guide are combined with three unsupervised outlier detectors: Isolation Forest, Local Outlier Factor, and an autoencoder. The data come from one institution with three study programs and cover 15 semesters, including 3,948 student-semester records, 348 courses, and 1,636 lecturer teaching activities. Because the data already satisfy the Feeder validation, anomalies are injected into a copy of the data, and detection is evaluated over 10 random injections. Rules detect explicit violations exactly but miss records that satisfy every threshold. The outlier detectors recover part of these records, and their performance depends on the entity: for teaching activities, Isolation Forest and Local Outlier Factor recover about two thirds of them, while for student-semester records the best detector recovers 39 percent. A combined detector reaches an area under the ROC curve of 0.982 for student-semester records. The results indicate that rule-based and machine learning detection are complementary for compliance checking of academic data.**

***Index Terms*—anomaly detection, regulatory compliance, higher education, PDDIKTI, outlier detection, data quality**

I. Introduction

Higher education institutions in Indonesia are required to implement internal quality assurance and to report its data and outputs to the national higher education database, PDDIKTI, at least once per semester [1]. The current regulation keeps PDDIKTI as the main source of data for quality assurance and makes each institution responsible for the correctness and accuracy of the data in it [2]. Institutions submit these data through the Feeder application, which applies a set of validation rules before synchronization [3]. The accuracy of the reported data affects accreditation, performance monitoring, and public accountability.

Data quality problems in administrative systems are well documented: errors enter through manual entry, system integration, and inconsistent interpretation of definitions, and they are costly to correct after they propagate [16], [17]. Studies of PDDIKTI reporting at Indonesian institutions likewise trace poor data quality to weak data quality management and have catalogued hundreds of business and data quality rules [18]. In national reporting, a single inconsistent record can distort ratios such as lecturer-to-student ratio or study duration, so institutions need a systematic way to find suspicious records before submission.

Rule-based validation catches violations that can be written down, such as a grade point average above 4.0 or a semester credit load above 24. It cannot catch records that satisfy every threshold but are unusual in combination, and it requires a new rule for every new type of error. Unsupervised outlier detection can complement rules because it needs no labeled violations, but its behavior on academic reporting data has received little systematic study.

This paper evaluates rule-based and machine learning detection of compliance anomalies using PDDIKTI data only. The questions are: (1) which violations can the Feeder validation rules detect, (2) how well do unsupervised detectors find records that satisfy all rules, and (3) does combining the two improve detection. The main contributions are a rule set derived from the Feeder guide and mapped to PDDIKTI attributes, an evaluation protocol based on injected anomalies, and an empirical comparison of rules, Isolation Forest, Local Outlier Factor, an autoencoder, and their combination.

The rest of the paper is organized as follows. Section III describes the data and rules, Section IV describes the methods and evaluation, Section V presents the results, and the last section concludes.

II. Related Work

A. Outlier Detection

Anomaly detection has been surveyed extensively, including deep approaches [7] and a unified view of deep and shallow methods [8]. Methods are commonly grouped into density-based, isolation-based, and reconstruction-based approaches. Isolation Forest isolates a record by random splits and uses the path length as an anomaly score [4]; Extended Isolation Forest replaces the axis-parallel splits with random hyperplanes to remove the bias this creates in the scores [9]. Local Outlier Factor compares the local density of a record with that of its neighbors, which allows it to find outliers relative to local structure [5]. Autoencoders learn to reconstruct typical records through a low-dimensional representation, and a large reconstruction error indicates an anomaly [7], [8]. These detectors are available in widely used libraries [6], [10].

B. Benchmarks and Ensembles

Large benchmarks on tabular data show that no unsupervised detector dominates: in the ADBench study of 30 algorithms on 57 datasets, no unsupervised algorithm statistically outperformed the others [11]. A recent re-evaluation of classical and deep detectors, including Isolation Forest and autoencoders, revisits whether tree-based methods can match deep ones [12], and interpretable tabular detectors have been proposed for settings where an auditor must understand a flag [13]. This variability motivates ensembles. Evaluations of heterogeneous unsupervised ensembles study how the normalization of detector scores affects the combination [14], and recent ensemble methods adjust each score using the scores of neighboring records [15]. The combination in this paper uses percentile ranks, which makes the three score scales comparable before averaging.

C. Data Quality and Rule-Based Checking

Data quality problems are a central obstacle for data-driven systems, and recent work treats data quality as a central concern of data-centric AI [16], [17]. In Indonesian higher education, explicit validation rules remain the standard tool: the Feeder application applies built-in checks [3], and an analysis at a statistics polytechnic derived 243 data quality rules and an assessment matrix for data reported to PDDIKTI [18]. Rules give exact and explainable results, but they cover only known violation types.

D. Anomaly Detection in Educational Data

Anomaly detection has been applied to aberrant student responses in intelligent tutoring systems with several unsupervised methods [19], and distance-based measures have been used to detect sudden changes in educational records [20]. To our knowledge, work that treats compliance of institutional reporting data with higher education regulations as an anomaly detection problem is scarce. This study therefore evaluates rules and unsupervised detectors on the same data and measures how much each contributes.

III. Data and Regulatory Rules

A. PDDIKTI Dataset

The data are replicated from the Feeder PDDIKTI web service into a relational store, one table per Feeder entity. Table I summarizes the entities used in this study.

TABLE I
PDDIKTI Entities and Attributes Used

Entity	Attributes used
Student biodata	name, birth place, birth date
Student education history	registration type, recognized credits, entry and exit period, entry and exit date
Student transfer grades	recognized course credits, recognized grade
Student semester activity	semester GPA, cumulative GPA, semester credits, total credits, status
Course	total credits, lecture, practicum, field practice, and simulation credits
Class	capacity, number of students, credits
Lecturer teaching activity	planned and realized meetings, substance credits
Study period	start and end date, number of meeting weeks, new-student target and re-registration counts

The extract covers one higher education institution with three study programs and 15 academic semesters (2020/1 to 2026/1). It contains 709 students (711 registrations), 3,948 student-semester records, 348 courses, 1,607 classes, 1,765 lecturer teaching activities by 90 lecturers, and 2,880 transfer-grade records.

B. Compliance Rules

The explicit rules are taken from the built-in validation of the Feeder PDDIKTI application [3]. Table II lists the rules that can be evaluated on PDDIKTI data alone.

TABLE II
Rule-Based Validation Criteria from the Feeder PDDIKTI Guide

Entity	Violation condition
Student	Exit period earlier than entry period; entry date not before graduation date; age under 10 years
Student	New admission with recognized credits > 0; transfer or degree-conversion admission with recognized credits $= 0$, or > 0 without transfer grades
Student activity	GPA > 4.0; semester GPA > 4.0; GPA $= 0$ while semester GPA > 0; total and semester credits both 0 while enrollment exists
Student activity	Semester credit load > 24
Course	Sum of lecture, practicum, field practice, and simulation credits differs from course credits, equals 0, or course credits > 9
Lecturer activity	Realized meetings > 18
Study period	Overlapping lecture periods within one study program
Student	Duplicate records by name, birth date, and birth place

All rules in Table II map to attributes available in the replicated schema, with one exception: the identity check

DRAFT

against the civil registry (DUKCAPIL) requires an external source and is excluded. The experiments cover three entities for which the rules are directly evaluable: student-semester records, courses, and lecturer teaching activities (1,636 records with planned and realized meetings).

IV. Methodology

The pipeline has three stages: rule-based detection, machine learning outlier detection, and evaluation on data with injected anomalies.

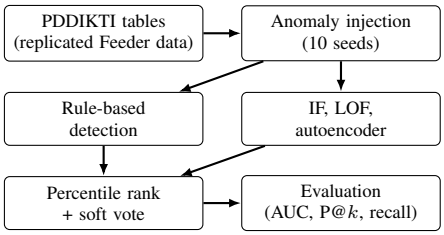


Fig. 1. Detection and evaluation pipeline. Injected anomalies are added to a copy of the replicated tables, and rules and ML detectors are scored against the injected labels.

A. Rule-Based Detection

Each rule in Table II is implemented as a predicate $r_k(x) \in \{0, 1\}$ over a record x of the relevant entity. A record is flagged when $\max_k r_k(x) = 1$, and the set of violated rules is kept as the explanation. Rules are evaluated per entity table, so no join with an external system is required.

B. Machine Learning Outlier Detection

Three unsupervised detectors are applied to numeric features derived from the same tables (Table III summarizes their characteristics):

- **Isolation Forest**, which isolates records by random axis-parallel splits; shorter average path length indicates a more anomalous record.
- **Local Outlier Factor (LOF)**, which compares the local density of a record with that of its k nearest neighbors.
- **Autoencoder**, a dense network trained to reconstruct normal records; the reconstruction error is the anomaly score.

Features are built per entity from the attributes in Table I. Student-semester records use semester GPA, cumulative GPA, semester credits, total credits, and an active-status indicator. Courses use total credits, the four credit components, and the difference between total credits and the component sum. Teaching activities use planned meetings, realized meetings, and their ratio. Let $x \in \mathbb{R}^d$ be the feature vector of a record. Isolation Forest scores x by $s_{\mathrm{IF}}(x) = 2^{-E[h(x)]/c(n)}$, where $E[h(x)]$ is the mean path length over the trees and $c(n)$ the average path length of an unsuccessful search in a binary tree of n points [4]. LOF scores x by the ratio of the mean local reachability density of its neighbors to its own density, so values near 1 are normal and larger values are anomalous [5]. The autoencoder scores x by the squared reconstruction error $s_{\mathrm{AE}}(x) = \|x - \hat{x}\|_2^2$. Features are standardized before fitting, and each detector is fitted separately for each entity on the unlabeled data. Isolation Forest uses 200 trees, LOF uses $k = 20$ neighbors, and the autoencoder is a multilayer perceptron with three hidden layers (sizes $2d$, $\lfloor d/2 \rfloor + 1$, and $2d$ for d input features) trained for at most 400 iterations; the autoencoder is implemented with scikit-learn [6].

TABLE III
Characteristics of the Detectors Compared

Detector	Principle	Suited to	Weakness
Rules	explicit predicate	known violations, exact and explainable	misses unknown types
Isolation Forest	random splits, path length	isolated global outliers	weak on anomalies inside dense regions
LOF	local density ratio	local deviations, clusters of different density	sensitive to k, $O(n^2)$ neighbor search
Autoencoder	reconstruction error	unusual feature combinations	needs enough data, unstable on small tables

C. Evaluation with Injected Anomalies

The extracted data already pass the Feeder validation, so almost no rule violations remain (Section III). Ground truth is therefore created by injecting anomalies into a copy of the data. For each rule in Table II, a fixed fraction of randomly chosen records is modified so that it violates that rule, for example setting cumulative GPA above 4.0, raising semester credits above 24, or setting an exit period before the entry period. Two further groups are injected to test the ML detectors beyond the explicit rules: records that stay within every rule threshold but deviate in combination (for example high credit load with very low GPA), and records with values shifted by a multiple of the attribute standard deviation. Injected records are labeled as anomalies and all others as normal. In total, 5% of the records of each entity are injected, split equally among the injection types of that entity (student-semester: four rule-based types and one subtle type; course: three rule-based types; teaching: one rule-based and one subtle type). Subtle anomalies are defined only for student-semester and teaching records. The injection is repeated with 10 random seeds.

D. Experimental Setup and Evaluation Metrics

Rule-based detection is evaluated by precision, recall, and F1-score against the injected labels. The ML detectors are evaluated by the area under the ROC curve (AUC) and Precision@k, with k equal to the number of injected anomalies. Results are reported as mean and standard deviation over the 10 injections. The rule-based method is scored by its binary flag. To make the three ML scores comparable, each is converted to its percentile rank $\rho_m(x) \in [0, 1]$ within the entity table, and the ML score of a record is the mean $\bar{\rho}(x) = \frac{1}{3} \sum_m \rho_m(x)$, an unweighted soft vote. A combined detector ranks records flagged by any rule first and orders the

DRAFT

rest by the mean percentile rank of the three ML scores; it flags a record when a rule fires or the record is among the top-k ML-ranked records. Precision@k is $|\{\text{top-}k\} \cap A|/k$ for the set A of injected anomalies, and the AUC is the probability that a randomly chosen anomaly receives a higher score than a randomly chosen normal record, the metric also used in recent benchmarks [11]. The recall of the top-k set is also reported separately for rule-based and subtle anomalies. Because the injected anomalies are synthetic, the results measure detection capability under controlled conditions rather than the prevalence of real violations.

TABLE IV
EXPERIMENTAL SETTINGS

Item	Setting
Isolation Forest	200 trees
LOF	$k = 20$ neighbors
Autoencoder	MLP, layers $2d$, $\lfloor d/2 \rfloor + 1$, $2d$; at most 400 iterations
Preprocessing	standardization per entity table
Score combination	mean percentile rank of the three ML scores
Injection rate	5% of records per entity, split equally among injection types
Repetitions	10 random injections
Reported metrics	precision, recall, F1 (rules); AUC, Precision@k, top-k recall (ML, combined)

E. Explainability of Findings

A flagged record is only useful to an auditor if the reason is visible. For rule-based detections the explanation is the set of violated rules, which refers directly to a criterion in Table II. For ML detections no rule exists, so the explanation is the feature vector of the record together with the three individual scores and their percentile ranks. A reviewer can then see, for example, that a record is unusual for the autoencoder but not for Isolation Forest, which usually points to an unusual combination of values rather than one extreme value. This distinction is useful because it tells the reviewer whether to look for a single wrong field or for an inconsistency between fields.

V. RESULTS AND DISCUSSION

No record in the original data violates any rule in Table II for the three entities studied. Table V reports AUC and Precision@k on the data with injected anomalies, and Table VI reports the recall of the top-k set by anomaly type.

By construction, the rules detect every rule-based injected anomaly with precision 1.0, because those anomalies are generated from the same rules; this is not evidence of accuracy on real violations. The informative results concern the subtle anomalies, which satisfy every rule. The rules flag essentially none of them; the small nonzero rules entries in Table VI come from the binary rule score, whose ties are broken arbitrarily when the top-k set is filled. The ML detectors recover a larger share, but the outcome depends on the entity: for teaching records, Isolation Forest and LOF recover about two thirds, while for student-semester records the best detector, the autoencoder, recovers 39%, and Isolation Forest recovers almost none. The combined detector gives the best AUC for student-semester records (0.982) because the rules rank explicit violations first, but it recovers little of the subtle anomalies in that entity.

TABLE V
AUC AND PRECISION@k (MEAN $\pm$ STD OVER 10 INJECTIONS)

Entity	Method	AUC	P@k
Student-semester	Rules	0.904 ± 0.004	0.810 ± 0.008
	Isolation Forest	0.826 ± 0.009	0.237 ± 0.044
	LOF	0.884 ± 0.015	0.386 ± 0.029
	Autoencoder	0.888 ± 0.033	0.515 ± 0.058
	Combined	0.982 ± 0.006	0.823 ± 0.018
Course	Rules	1.000 ± 0.000	1.000 ± 0.000
	Isolation Forest	0.983 ± 0.005	0.700 ± 0.092
	LOF	0.993 ± 0.006	0.906 ± 0.027
	Autoencoder	0.908 ± 0.097	0.506 ± 0.173
	Combined	1.000 ± 0.000	1.000 ± 0.000
Teaching	Rules	0.750 ± 0.000	0.513 ± 0.016
	Isolation Forest	0.979 ± 0.005	0.818 ± 0.041
	LOF	0.865 ± 0.037	0.785 ± 0.036
	Autoencoder	0.872 ± 0.077	0.420 ± 0.199
	Combined	0.971 ± 0.012	0.820 ± 0.043

TABLE VI
TOP-k RECALL BY ANOMALY TYPE (MEAN OVER 10 INJECTIONS)

Entity	Method	Rule-based	Subtle
Student-semester	Rules	1.000	0.049
	Isolation Forest	0.293	0.015
	LOF	0.405	0.308
	Autoencoder	0.547	0.390
	Combined	1.000	0.113
Teaching	Rules	1.000	0.027
	Isolation Forest	0.976	0.661
	LOF	0.902	0.668
	Autoencoder	0.695	0.144
	Combined	1.000	0.639

A. Analysis by Entity

Student-semester records. The rules reach an AUC of 0.904 because they rank the rule-based injections (four fifths of the injected anomalies) perfectly, while the subtle fifth is tied with the normal records, which gives $0.8 + 0.2 \times 0.5 = 0.9$. Among the ML detectors the autoencoder is the best on this entity (AUC 0.888, Precision@k 0.515), followed by LOF (0.884) and Isolation Forest (0.826). Isolation Forest recovers only 1.5% of the subtle anomalies. A plausible reason is that a subtle record, such as a high credit load with a low GPA, has ordinary values on each single attribute, so random axis-parallel splits isolate it no faster than normal records; detectors that look at the joint structure do better. The combined detector raises the AUC to 0.982 because the rule flags push explicit violations to the top and the ML ranks order the rest.

DRAFT

Courses. Rules, and therefore the combined detector, are perfect on this entity because all injected anomalies are rule-based and each violates an arithmetic condition on the credit components. LOF is the strongest ML detector (AUC 0.993), and the autoencoder shows a large standard deviation (0.097) that is consistent with the small table of 348 records. This entity illustrates that a rule written for a known arithmetic constraint is hard to improve on, and that ML adds little when the violated constraint is already encoded.

Teaching activities. This is the entity where the ML detectors matter most. The rules reach an AUC of 0.750, but Isolation Forest reaches 0.979 and the combined detector 0.971. Isolation Forest and LOF recover about two thirds of the subtle anomalies (0.661 and 0.668), whereas the autoencoder recovers 14%. The ratio of realized to planned meetings is likely a strong low-dimensional signal, which would suit isolation- and density-based detectors better than a reconstruction model with few input features, although this explanation was not tested. The combined detector is slightly below Isolation Forest in AUC here, which shows that an unweighted vote can be dragged down by a weaker member; weights tuned on labeled cases are a natural extension.

These results indicate that rule-based and ML detection are complementary: rules provide exact, explainable detection of explicit violations, and the unsupervised detectors cover deviations that no rule describes. The conclusions are limited by the controlled setting. Subtle anomalies are defined by the authors, the data come from a single institution with three study programs, and the course entity has only 348 records, which explains the large variance of the autoencoder. Validation on naturally occurring violations remains future work.

B. Practical Implications

The results suggest a division of labor in a compliance workflow. Rules should remain the first line of defense because they are exact, cheap, and cite the governing criterion, which is what an auditor needs to act on a finding. The unsupervised detectors are best used as a second stage that ranks the records no rule describes, so that reviewers inspect the most unusual records first instead of the whole table. Because the detectors disagree on which subtle records are unusual, especially for student-semester data, showing the individual detector scores next to the combined score gives the reviewer more information than a single number. Detection can also be run per semester so that each record is compared with its contemporaries and not with the whole history, which avoids flagging records only because academic conditions changed over time.

C. Threats to Validity

Three limitations affect how far the results can be generalized. First, the anomalies are synthetic: rule-based injections are generated from the rules themselves, and subtle anomalies reflect the authors' assumptions about what an inconsistent record looks like, so the numbers describe detection capability and not the prevalence of real violations. Second, the data come from a single institution with three study programs, and the course entity has only 348 records, which makes the autoencoder unstable. Third, the detectors use default or lightly tuned hyperparameters; no search over the number of neighbors, the number of trees, or the network size was performed, and a different choice could change the ranking among detectors. Averaging over 10 random injections reduces the effect of a single draw, but it does not remove these biases.

VI. Conclusion

This paper evaluated rule-based and machine learning detection of compliance anomalies on PDDIKTI data using rules from the Feeder validation guide and injected anomalies. Rules detect explicit violations exactly but miss deviations that satisfy all thresholds, while Isolation Forest, LOF, and an autoencoder recover part of those deviations to an extent that depends on the entity. The combined detector gives the best overall ranking when explicit violations dominate, while the choice of unsupervised detector matters most for subtle anomalies. Future work includes validation on naturally occurring violations and additional institutions, the remaining entities such as enrollment history and study periods, per-semester scoring with weighted voting tuned on labeled cases, and linking each flagged record to the text of the regulation it may violate so that findings are explained to auditors in natural language, for example with retrieval-augmented generation [21].

DRAFT

Acknowledgment

References

[1] Kementerian Pendidikan, Kebudayaan, Riset, dan Teknologi, "Peraturan Menteri Pendidikan, Kebudayaan, Riset, dan Teknologi Nomor 53 Tahun 2023 tentang Penjaminan Mutu Pendidikan Tinggi," Berita Negara Republik Indonesia Tahun 2023 Nomor 638, Aug. 2023 (revoked by [2]).

[2] Kementerian Pendidikan Tinggi, Sains, dan Teknologi, "Peraturan Menteri Pendidikan Tinggi, Sains, dan Teknologi Nomor 39 Tahun 2025 tentang Penjaminan Mutu Pendidikan Tinggi," Berita Negara Republik Indonesia Tahun 2025, promulgated 25 Sep. 2025.

[3] Kementerian Pendidikan dan Kebudayaan, "User guide UI/UX Feeder PDDIKTI, version 4.0," Direktorat Jenderal Pendidikan Tinggi, Dec. 2020.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[5] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 93–104.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[7] G. Pang, C. Shen, L. Cao, and A. van den Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 38:1–38:38, Mar. 2022, doi: 10.1145/3439950.

[8] L. Ruff *et al.*, "A unifying review of deep and shallow anomaly detection," *Proc. IEEE*, vol. 109, no. 5, pp. 756–795, 2021.

[9] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, Apr. 2021, doi: 10.1109/TKDE.2019.2947676.

[10] S. Chen *et al.*, "PyOD 2: A Python library for outlier detection with LLM-powered model selection," arXiv:2412.12154, 2024.

[11] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Proc. 36th Conf. Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[12] S. Mehta, S. Mehendale, N. Fernandes, J. Sarkar, S. Sarkar, and S. Saha, "Benchmarking anomaly detection algorithms: Deep learning and beyond," arXiv:2402.07281, 2024.
[13] C.-H. Chang, J. Yoon, S. Arik, M. Udell, and T. Pfister, "Data-efficient and interpretable tabular anomaly detection," in *Proc. 29th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 190–201, doi: 10.1145/3580305.3599294.
[14] S. Klüttermann and E. Müller, "Evaluating and comparing heterogeneous ensemble methods for unsupervised anomaly detection," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2023, pp. 1–8, doi: 10.1109/IJCNN54540.2023.10191405.
[15] J. Yang, S. Rahardja, and S. Rahardja, "Regional ensemble for improving unsupervised outlier detectors," *IEEE Trans. Artif. Intell.*, vol. 5, no. 9, pp. 4391–4402, Sep. 2024, doi: 10.1109/TAI.2024.3381102.
[16] S. E. Whang, Y. Roh, H. Song, and J.-G. Lee, "Data collection and quality challenges in deep learning: A data-centric AI perspective," *VLDB J.*, vol. 32, no. 4, pp. 791–813, 2023, doi: 10.1007/s00778-022-00775-9.
[17] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, and X. Hu, "Data-centric artificial intelligence: A survey," *ACM Comput. Surv.*, vol. 57, no. 5, pp. 129:1–129:42, May 2025, doi: 10.1145/3711118.
[18] A. R. Sumadi and N. Wilantika, "Analisis kebutuhan kualitas data dan aturan bisnis data pendidikan," *Sistemasi: Jurnal Sistem Informasi*, vol. 10, no. 3, pp. 726–740, Sep. 2021, doi: 10.32520/stmsi.v10i3.1381.
[19] G. Gorgun and O. Bulut, "Identifying aberrant responses in intelligent tutoring systems: An application of anomaly detection methods," *Psychol. Test Assess. Model.*, vol. 64, no. 4, pp. 359–384, Dec. 2022.
[20] A. D. Smith, "Event detection in educational records: An application of big data approaches," *Int. J. Bus. Syst. Res.*, vol. 15, no. 3, pp. 271–291, 2021, doi: 10.1504/IJBSR.2021.114936.
[21] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

DRAFT